

Reviewer Pack — Minimal Tropicalization of Automorphism Groups and DPLL Proo...

Entry a023d9562a86 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 17:21:47 UTC

Minimal Tropicalization of Automorphism Groups and DPLL Proof Path Length

Entry ID: a023d9562a86 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 17:21:47 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Path Length)

Statement:

For any given Boolean satisfiability instance φ with n variables, the minimal order of tropical automorphism groups associated with the tropicalization of φ is linearly correlated with its DPLL proof path length, such that $\text{ord}_t(\varphi) = \Theta(p(\varphi))$, where $p(\varphi)$ is a function of the DPLL proof path length.

Rationale (proposer's reasoning):

Tropical geometry offers a way to study structures in a simpler setting while preserving certain properties. The automorphism groups of these tropicalized objects could reveal underlying combinatorial structures related to the complexity of solving Boolean satisfiability problems. A connection between the group order and proof length might expose non-trivial features of the problem's structure.

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 761a019e1283c463

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

To support our conjecture, we must find that for at least 80% of the 30 randomly generated Boolean satisfiability instances with n variables ($n \leq 40$), the minimal order of tropical automorphism groups $\text{ord}_t(\varphi)$ associated with their tropicalization φ_{trop} is within a factor of 10 of the DPLL proof path length $p(\varphi)$, measured as $|\text{ord}_t(\varphi) - p(\varphi)| \leq 3 * \max(p(\varphi))$ across all instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "Boolean satisfiability" AND DPLL proof path length - "minimal tropicalization" AND automorphism groups AND DPLL proof path - "linear correlation" AND $\text{ord}_t(\phi) = \theta(p(\phi))$ AND Boolean satisfiability

Top relevant hits considered: - [http://arxiv.org/abs/physics/0604066v2] Boolean Game on Scale-free Networks - [http://arxiv.org/abs/2412.20291v1] Efficient Learning and Computation of Linear Correlated Equilibrium in General Convex Games - [http://arxiv.org/abs/1806.03069v2] Heavy-ion double-charge-exchange and its relation to neutrinoless double-beta decay

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_instance(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropicalize(instance):
        n = len(instance)
        trop_points = []
        for i in range(2**n):
            point = []
            for j in range(n):
                if instance[i] & (1 << j):
                    point.append(j + 1)
                else:
                    point.append(-j - 1)
            trop_points.append(point)
        return trop_points

    def order_of_automorphism_group(trop_points):
        n = len(trop_points[0])
        symmetries = []
```

```

    for i in range(2**n):
        permuted_points = [trop_points[(i >> j) & 1] for j in range(n)]
        if all(sorted(p) == sorted(q) for p, q in zip(trop_points, permuted_points)):
            symmetries.append(i)
    return len(symmetries)

def dpll_path_length(instance):
    n = len(instance)
    stack = []
    path_length = 0
    while stack or any(x == 1 for x in instance):
        if not stack:
            stack.append((instance, 0))
            path_length += 1
            current_instance, pos = stack[-1]
        if pos == n:
            stack.pop()
            continue
        if current_instance[pos] == 0:
            new_instance = current_instance[:]
            new_instance[pos] = 1
            stack.append((new_instance, pos + 1))
            path_length += 1
        else:
            stack[-1] = (current_instance, pos + 1)
    return path_length

n = random.randint(5, 40)
instance = generate_boolean_instance(n)
trop_points = tropicalize(instance)
order_t = order_of_automorphism_group(trop_points)
path_length = dpll_path_length(instance)

return {
    "metric_name": "order_t - path_length",
    "metric_value": abs(order_t - path_length),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": abs(order_t - path_length) <= 3 * max(path_length),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29,
        31, 37, 41, 43, 47, 53, 59, 61, 67, 71,
        73, 79, 83, 89, 97, 101, 103, 107, 109, 113
    ]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)

```

```

std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support condition of finding that at least 80% of the instances meet the specified criteria.
| next: NONE

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16023
2	propose	ollama_remote	glm4:latest	0	0	12118
3	propose	ollama_remote	glm4:latest	0	0	13162
4	preregistration	ollama_remote	glm4:latest	0	0	9629
5	novelty	ollama_remote	glm4:latest	0	0	15271
6	novelty	ollama_remote	glm4:latest	0	0	9339
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19657
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9006
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12067
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10782
11	judge	ollama_remote	glm4:latest	0	0	15820

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 142874 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/a023d9562a86.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a023d9562a86.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a023d9562a86.tar.gz` (if generated)